



implemented. For example, ObservationPack originates from two context hypotheses but ultimately modifies the observation boundary. This distinction helps organize the search while leaving the architecture of each mechanism open. Each independent search follows the conventional autoresearch cycle: propose a change, implement it, run a fixed experiment, inspect the result, and retain, revise, or discard the candidate [23]. We extend this cycle with an iterative implementation loop based on the Ralph Loop [24]. The implementer refines the candidate to meet an explicit completion criterion, then an independent reviewer checks it before evaluation. Failed reviews trigger revision. Each iteration may yield multiple exploration trajectories. Independent analyzers examine one trajectory each for repeated action context growth, large observations, or sparse diagnostic signals. A reducer combines their findings into a candidate-level summary that guides the next proposal. Figure 2 summarizes the workflow and its isolation boundary.

We run each independent search as a disposable instance of a shared skill template containing a minimal research local and operating instructions. Each experiment copies the template, sets its parameters, and runs to completion, retaining the candidate and evidence while discarding modified orchestration code. Search breadth comes from running multiple isolated loops; depth comes from repeated refinement within each loop. These counts describe the scope of our search; they do not establish a scaling law.

3. Search Environments

Our search pipeline uses separate environments for mechanism discovery and transfer evaluation. The search space comprises 535 executable environments: 495 repository tasks derived from GitHub issue–pull request pairs and 40 synthetic tasks with executable success verifiers. Together, they support automatically evaluated search grounded in real

software changes and open-ended problem solving.

Repository-derived environments. Each environment pairs a GitHub issue with its pre-fix repository state and offline dependencies, using the accepted patch and change history as a reference trajectory. The pull request and regression test are hidden from the agent. We retain only environments whose test fails before the patch and passes afterward.

Verifier-driven environments. We first generate an executable verifier that defines success, then construct a task environment around it. This allows multiple valid solution paths without requiring a reference trajectory. The 40 synthetic environments primarily use a Terminal-Bench-2-style verifier interface [25], extending the search beyond repository histories while preserving automatic evaluation. Figure 3 summarizes both construction paths and the isolation boundary between mechanism search and held-out evaluation.



Figure 3 | Two environment families support mechanism discovery while keeping EdgeBench held out. Repository-derived environments use pre-fix repositories and hidden regression tests verified to fail before the accepted patch and pass afterward. Verifier-driven environments allow multiple solution paths under executable success criteria. EdgeBench remains isolated from search and is used only after the harness is frozen.